

CS 331 – Database Systems
Assignment #3 – Functional Dependencies
Arnav Kucheriya, Jennifer Corte Juela

1. (20%) What update anomalies could occur in the following relation:

BOOK_PUBLISHER

<u>Book_id</u>	Title	Publisher_Name	Publisher_Address	Publisher_Phone
----------------	-------	----------------	-------------------	-----------------

- Insert Anomaly:
 - Occurs when you cannot record data about one entity because the other entity is missing
 - Cannot add a new Publisher to the database until they have published at least one Book.
 - Book_id is the primary key, so it cannot be NULL. If a publisher hasn't published a book yet, there is no valid way to enter their name, address, or phone number into the table.
- Deletion Anomaly:
 - Occurs when deleting one piece of data results in the unintentional loss of unrelated data.
 - If the only book associated with a specific publisher is deleted, all contact information (address and phone) for that publisher is permanently deleted from the database.
- Update Anomaly:
 - Occurs when data is stored redundantly, making it difficult to keep the information consistent.
 - If a publisher changes their phone number, every single row (every book) must be updated associated with that publisher.
 - If a publisher moves to a new address, every single book must be updated associated with that publisher.
 - Since the publisher's address would be repeated for every book they have in the table, failing to update even one row leads to data inconsistency, where the same publisher appears to have two different addresses in different parts of the table.
 - For example, if a publisher has 500 books listed, you must perform 500 updates.

2. (30%) Consider a relation R(A,B,C,D,E,F,G,H,I,J) with the following functional dependencies:

AB → C

BD → EF

AD → GH

A → I

H → J

a) Find all candidate keys of R.

Identify attributes that never appear on the right-hand side of any functional dependency.

Right-hand side attributes are:

C, E, F, G, H, I, J

So the attributes A, B, and D do not appear on the right-hand side.

Compute the closure of {A, B, D}:

- From $AB \rightarrow C$, get C
- From $BD \rightarrow EF$, get E and F
- From $AD \rightarrow GH$, get G and H
- From $A \rightarrow I$, get I
- From $H \rightarrow J$, get J

$(ABD)^+ = \{A, B, C, D, E, F, G, H, I, J\}$

Thus, ABD determines all attributes.

Test minimality:

- Remove A: BD^+ gives B, D, E, F only, not all attributes
- Remove B: AD^+ gives A, D, G, H, I, J only, not all attributes
- Remove D: AB^+ gives A, B, C, I only, not all attributes

So ABD is minimal.

Candidate key = {A, B, D}

b) Normalize R into 2NF.

A relation is in 2NF if it is in 1NF and has no partial dependency of a non-prime attribute on part of a candidate key.

Candidate key: ABD

Identify partial dependencies on subsets of ABD:

- $AB \rightarrow C$
C depends on part of the key
- $BD \rightarrow EF$
E and F depend on part of the key
- $AD \rightarrow GH$
G and H depend on part of the key
- $A \rightarrow I$
I depends on part of the key

Also:

- $H \rightarrow J$ is a transitive dependency, which is allowed in 2NF

Decompose by removing the partial dependencies:

2NF Relations:

1. $R_1(A, B, C)$
 - a. Key: AB
2. $R_2(B, D, E, F)$
 - a. Key: BD
3. $R_3(A, D, G, H, J)$
 - a. Key: AD
 - b. J is included here because 2NF does not yet remove transitive dependencies
4. $R_4(A, I)$
 - a. Key: A
5. $R_5(A, B, D)$
 - a. Key: ABD
 - b. This preserves the original candidate key relationship

2NF decomposition:

- $R_1(A, B, C)$
- $R_2(B, D, E, F)$
- $R_3(A, D, G, H, J)$
- $R_4(A, I)$
- $R_5(A, B, D)$

c) Normalize R into 3NF.

A relation is in 3NF if it is in 2NF and has no transitive dependencies of non-key attributes on a key.

From the original dependencies, the transitive dependency is:

- $AD \rightarrow H$
- $H \rightarrow J$
- therefore $AD \rightarrow J$ transitively through H

Reach 3NF, separate J from the relation containing H.

3NF Relations:

1. R1(A, B, C)
Key: AB
2. R2(B, D, E, F)
Key: BD
3. R3(A, D, G, H)
Key: AD
4. R4(A, I)
Key: A
5. R5(H, J)
Key: H
6. R6(A, B, D)
Key: ABD

3NF decomposition:

- R1(A, B, C)
- R2(B, D, E, F)
- R3(A, D, G, H)
- R4(A, I)
- R5(H, J)
- R6(A, B, D)

3. (20%) Consider the following relation:

CAR_SALE

<u>Car_id</u>	Date_sold	<u>Salesman_id</u>	Commision%	Discount_amount
---------------	-----------	--------------------	------------	-----------------

Assume that a car may be sold by multiple salesmen and hence {Car_id, Salesman_id} is the primary key.

Additional functional dependencies are:

- Date_sold $\rightarrow$ Discount_amount
- Salesman_id $\rightarrow$ Commission%
- Car_id $\rightarrow$ Date_sold

a) Normalize CAR_SALE into 2NF

Primary key: {**Car_id, Salesman_id**}

To be in 2NF, non-key attributes must depend on the whole key, not just part of it.

Check dependencies:

- Salesman_id $\rightarrow$ Commission%
Partial dependency on part of the key
- Car_id $\rightarrow$ Date_sold
Partial dependency on part of the key

Also:

- Since Car_id $\rightarrow$ Date_sold and Date_sold $\rightarrow$ Discount_amount, we get Car_id $\rightarrow$ Discount_amount, so Discount_amount also depends only on part of the key

Remove the partial dependencies.

2NF Relations:

1. CAR_SALE(Car_id, Salesman_id)
Key: (Car_id, Salesman_id)
2. CAR_INFO(Car_id, Date_sold, Discount_amount)
Key: Car_id
3. SALESMAN(Salesman_id, Commission%)
Key: Salesman_id

2NF decomposition:

- CAR_SALE(Car_id, Salesman_id)
- CAR_INFO(Car_id, Date_sold, Discount_amount)
- SALESMAN(Salesman_id, Commission%)

b) Normalize CAR_SALE into 3NF

In CAR_INFO(Car_id, Date_sold, Discount_amount):

- Car_id $\rightarrow$ Date_sold
- Date_sold $\rightarrow$ Discount_amount

So:

- Car_id $\rightarrow$ Discount_amount transitively through Date_sold

This violates 3NF.

So split CAR_INFO further.

3NF Relations:

1. CAR_SALE(Car_id, Salesman_id)
Key: (Car_id, Salesman_id)
2. CAR(Car_id, Date_sold)
Key: Car_id
3. SALE_DATE(Date_sold, Discount_amount)
Key: Date_sold
4. SALESMAN(Salesman_id, Commission%)
Key: Salesman_id

3NF decomposition:

- CAR_SALE(Car_id, Salesman_id)
- CAR(Car_id, Date_sold)
- SALE_DATE(Date_sold, Discount_amount)
- SALESMAN(Salesman_id, Commission%)

4. (30%) Consider the following relation:

MEDICAL

Physician_id	Patient_id	Date	Diagnosis	Treatment_Code	Charge
--------------	------------	------	-----------	----------------	--------

A tuple describes a visit of a patient to a doctor along with a treatment code and charge. Assume that a diagnosis is determined (uniquely) for each patient by a doctor and that each treatment code has a fixed charge (regardless of the patient).

a) Infer a set of (two) functional dependencies for MEDICAL from this description.

From the description:

1. A diagnosis is uniquely determined for each patient by a doctor:

$(\text{Physician_id}, \text{Patient_id}) \rightarrow \text{Diagnosis}$

2. Each treatment code has a fixed charge:

$\text{Treatment_Code} \rightarrow \text{Charge}$

The two functional dependencies are:

- $(\text{Physician_id}, \text{Patient_id}) \rightarrow \text{Diagnosis}$
- $\text{Treatment_Code} \rightarrow \text{Charge}$

Assumed primary key for normalization

Since each tuple represents a visit and includes a treatment code, a reasonable key for the original relation is:

$(\text{Physician_id}, \text{Patient_id}, \text{Date}, \text{Treatment_Code})$

This allows multiple visits on different dates and different treatments.

b) Normalize MEDICAL into 2NF

A relation is in 2NF if there are no partial dependencies on part of the composite key.

Primary key:

(Physician_id, Patient_id, Date, Treatment_Code)

Now check the dependencies:

- (Physician_id, Patient_id) → Diagnosis
Diagnosis depends only on part of the composite key
- Treatment_Code → Charge
Charge depends only on part of the composite key

These are partial dependencies, so we decompose.

2NF Relations:

1. VISIT(Physician_id, Patient_id, Date, Treatment_Code)
Key: (Physician_id, Patient_id, Date, Treatment_Code)
2. DOCTOR_PATIENT(Physician_id, Patient_id, Diagnosis)
Key: (Physician_id, Patient_id)
3. TREATMENT(Treatment_Code, Charge)
Key: Treatment_Code

2NF decomposition:

- VISIT(Physician_id, Patient_id, Date, Treatment_Code)
- DOCTOR_PATIENT(Physician_id, Patient_id, Diagnosis)
- TREATMENT(Treatment_Code, Charge)

c) Normalize MEDICAL into 3NF

- In VISIT, there are no non-key attributes
- In DOCTOR_PATIENT, Diagnosis depends directly on the key
- In TREATMENT, Charge depends directly on the key

So the 2NF decomposition is already in 3NF.

3NF decomposition:

- VISIT(Physician_id, Patient_id, Date, Treatment_Code)
- DOCTOR_PATIENT(Physician_id, Patient_id, Diagnosis)
- TREATMENT(Treatment_Code, Charge)